

“我们想找个质量评价标准, 打磨了一年, 从网上搜到这个代码缺陷率标准就用了:

目的

- 通过数据督促开发提高代码质量, 提高产品质量。

CMMI 定义

CMMI 级	千行代码缺陷率
动作 (Motion)	任务替换 (Task Switching, Multi-tasking)
1 级	11.95‰
2 级	5.52‰
3 级	2.39‰
4 级	0.92‰
5 级	0.32‰

千行代码缺陷率 = (bug 数 / 代码行数) * 1000‰

目的: 希望有些具体量化目标, 促进团队提升。请教你的意见?” 认识多年的质量经理问。

你们一直有系统管理缺陷, 应该从历史数据看到不同产品线, 对应不同市场, 和团队能力不同, 有不同的缺陷率范围。(假定说是内部系统测试阶段, 同样也适用于验收测试阶段。)

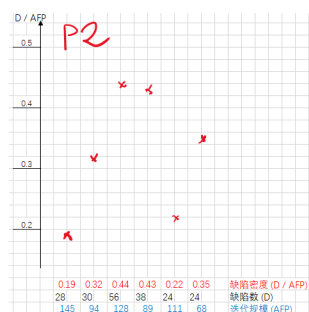
对于某些优秀团队, 他们可能已经超越了这个目标; 但某些初级的团队会难以达到。

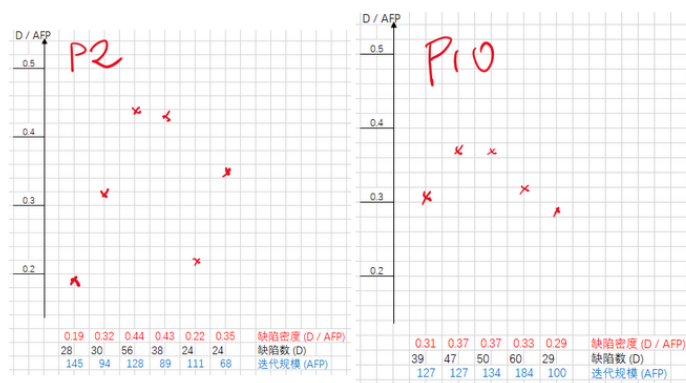
某些项目经理会想: “公司总部质量部制定的目标必须达到, 达不到怎么办?” 导致出现一年前跟你讨论过的情景:

。。。我们很注重量化质量管理, 搞度量分析, 但大家制定算法、原理就开始作假。现在我们主管对度量很反感。。。

更重要的应是让团队从过去迭代的缺陷数据统计现在系统能力范围, 如果系统已经稳定, 如果没有变化, 系统不会达到超越现在能力范围的目标。

下面是两个项目的迭代缺陷密度趋势图, 请问你觉得那个项目更合适制定量化改进目标?





质量经理答：“右面 P10 应该可以，左面 P2 的数据太散了。”

所以如果现在系统不稳定，没有能力，公司制定目标也起不来促进作用：如果公司目标与当前水平差很远，团队会放弃，或报些假数据应付；如果很容易达到，便会反而妨碍了团队自定更高的目标。

所以戴明博士的经典 14 点里包括“取消量化目标”这条：

“Eliminate numerical quotas for the workforce and numerical goals for management.”

建立基线

戴明博士每次质量演讲都会介绍如何使用控制图：

假设一家生产矿泉水的企业（通常一瓶水 500 毫升），为了确保每次机器装的水不会太多，也不能太少，每一轮生产都会随机抽 5 个样本，然后记录抽样的均值和范围（最大和最小）。

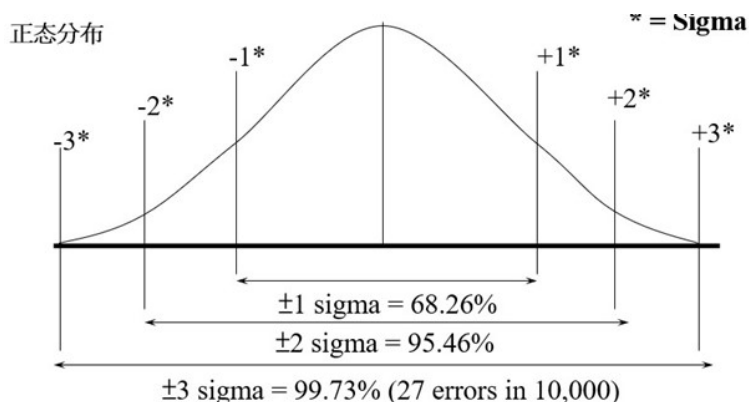
下图是过去 22 天均值的变化：



肯定是后面比之前的下跌了，但是如果等到最后 D 点才行动就太晚了，已经跌了太多，但是在那个点开始有变？

这也是 20 年代美国电话公司要解决的问题：很多电话线路与设备都埋在地下，调配维修工作很困难，如何能尽量减少无用的调配工作？

公司也利用生产线大量生产电话设备，需要调试生产线的各种设备参数，但工程师发现难以把生产线调试到稳定状态，如果按系数低了就调高，或者高了就调低，反而会越调越乱。针对这难题，Dr Shewhart（当时在 Bell Labs 工作）发明了控制图，用来区分是过程的噪音还是信号。下面是统计学的正态分布图：



正态分布，正负 3 个标准差包括了 99% 以上的面积。按中心极限定理，任何分布如果随机抽样够多的话，它的均值就是接近正态分布。所以控制图按样本平均值的标准差，用方程式计算上限和下限，如果后面有些点超出了上下限范围（因上下限是按正负 3 个标准差出来，所以任何后面的点超出这范围，它发生的概率是少于 1%。）利用控制图帮我们区分那些波动是噪音，那些波动（异常点）是信号。（参考附件，了解各种常用于生产管理的控制图，和中心极限定理。）注意：虽然上面用正态分布来理解为什么选上下 3 个标准差为控制图上下限，但 Shewhart 先生的控制图方法没有规定或假定数据必须是正态分布。

控制图的应用例子

看下面 1987 ~ 1988 美国国家总贸易逆差每月的数据趋势图，就更好了解美国贸易逆差这两年的变化。

美国按月贸易逆差，1987 ~ 1988 (\$, 亿美元)

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1987	10.7	13.0	11.4	11.5	12.5	14.1	14.8	14.1	12.6	16.0	11.7	10.6
1988	10.0	11.4	7.9	9.5	8.0	11.8	10.5	11.2	9.2	10.1	10.4	10.5

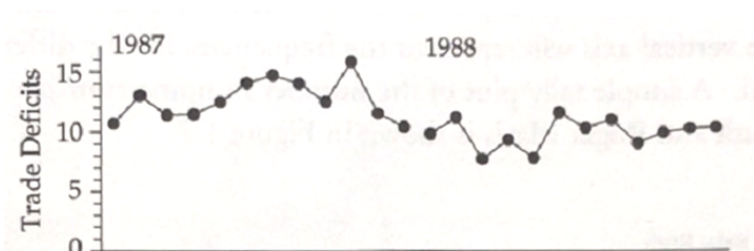


Figure 1.5: Monthly U.S. Trade Deficits 1987-1988

2 年的美国贸易

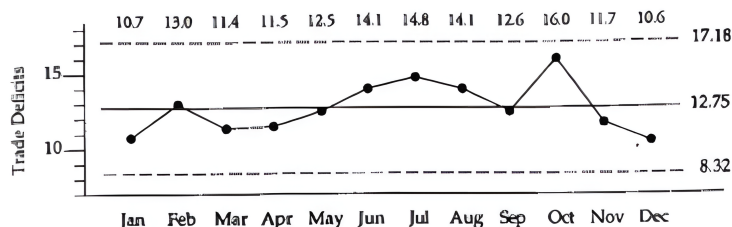
易逆差数据有变化吗？

有些人会觉得有变化，比如中间 1987 年 10 月就特别高，后面 1988 年的 5 月也特别高，应该是不正常。

但其他人可能看不出来，反而觉得好像是后面那些数比前面低了。

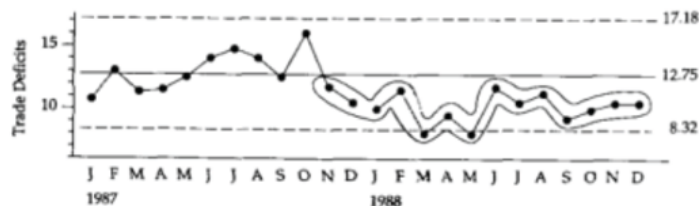
要判断分析数据有没有变化，很难单靠主观识别，需要有工具帮助。控制图便可发挥作用。

不是说 10 月份特别高吗，但是如果用控制图发现它还是在范围之内。如果我们多看两个月——11 月和 12 月，发现贸易逆差又跌下来。



我们继续看，

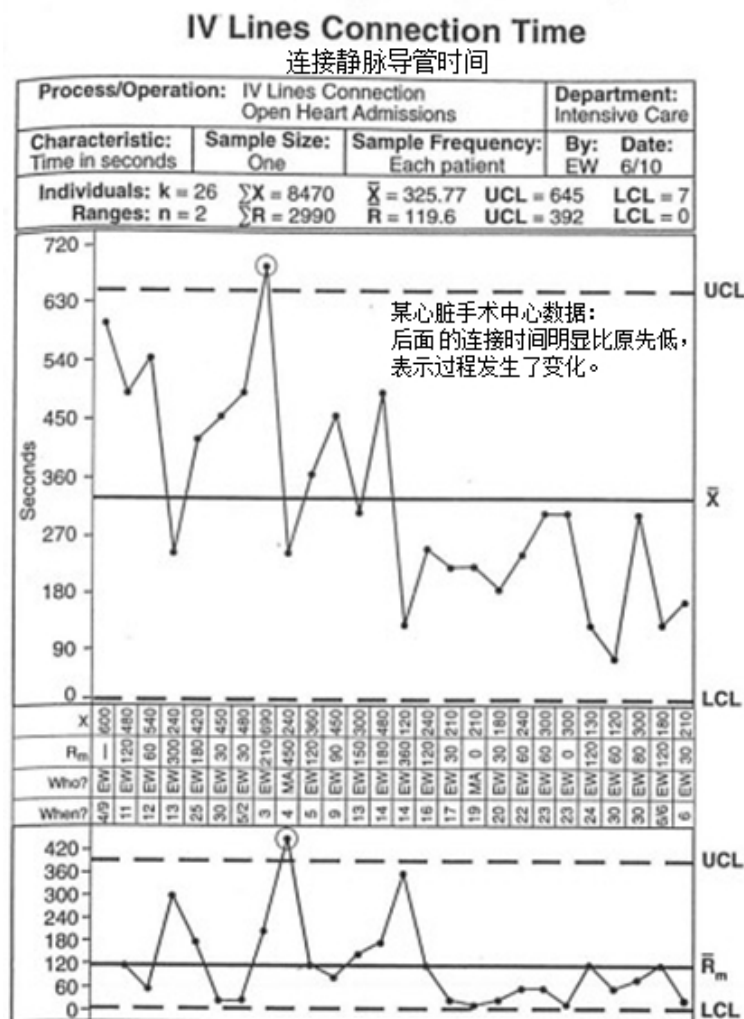
发现 3 月有异常点，后面在 5 月份也有异常点。如果我们再看后面月份的逆差数据，看见明显比前面降低。



控制图帮助我们识别数据的变化是噪音还是信号。例如 87 年 10 月份的 16.0 只是噪音（未超出上下线），但后面 88 年的 3 月份 7.9 和 5 月份的 8.0 是信号，表示逆差有根本改变，应重新按数据制定上下限。注意：不应把两年的数据的均值与范围（极差）作为基线，应该是以后面降低后的那些点，才能真正反应当前的水平和分布。

如果不是工业生产，不一定有这么多数据去随机抽样、求均值，怎么办呢？可以

用 ImR 图 (或 XmR), 比如下图就是某个医院做外科心脏手术所花的时间的统计, 想看看开心手术的时间是否稳定, 因为外科手术特别危险, 越长时间那患者的存活机会就越低, 也看到是有异常点。



(ImR 控制图相关公式, 详见附件)

ImR 控制图可以用于软件开发。比如一些已经进了维护期的产品都很稳定, 你就可以用控制图来识别有没有变化。但是一些全新的过程可能变化很大, 可能控制图就不一定适用。但如果是客服收多少投诉, 每天都是很稳定、连续的, 应该可以用。

软件开发应用实例

某企业非常关注交付有没有延误，每月都收集各个部门发生投产延误的次数，除以项目发布投产总数，统计并每月汇报高层。

从 2021 年初开始使用控制图，计算上限与下限 (0.98, 0.93)，3 月份统计发现系数是 0.92 (0.92 是按时交付率，所以是越少越差)，超出了下线范围，改进小组启动根因分析。

每月的统计数是从 7 个模块（事业部）汇总来，改进小组利用 80/20 原则，识别出贡献最大的头两个模块已占总数之 73%，便专门调研这两个模块。使用问卷、面谈等，找出主因：

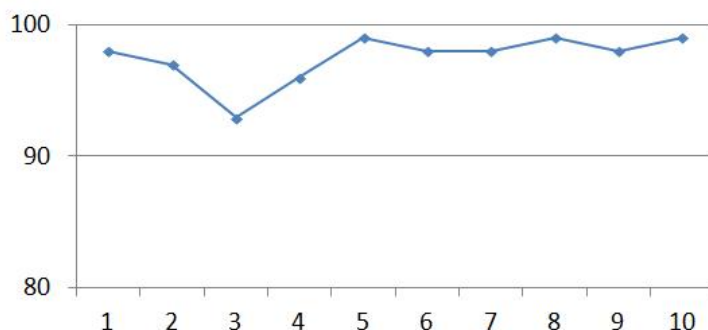
1. 新进项目经理较多，对里程碑管理、关闭条件等要求不清楚。
2. 项目有点难，和用户沟通效果不佳。

对应改进行动：

- 对项目经理的里程碑管理过程进行改进。包括：
 1. 培训（讲解项目管理工具的使用和里程碑管理要求）。
 2. 考核（明确如果出现里程碑红灯的考核标准）。
 3. 预防（建立专项沟通小组，对存在进度风险的项目，提醒项目经理尽快推进，项目经理和用户提前沟通，准备变更）。

我问：效果如何？

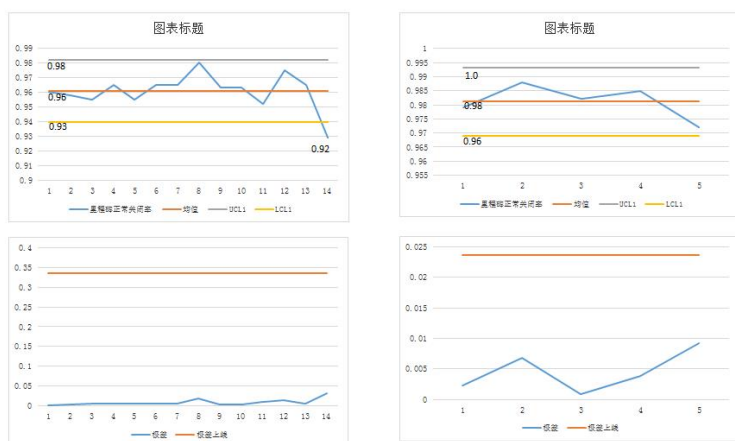
过程改进组长：请看下图今年到 10 月份的趋势图，从 3 月份的低点后面就明显改善了。



我：建议你用控制图方程式计算，从 5 月份到现在的统计图。

第二天，过程改进组长展示之前之后的两个控制图：

下面左图是 3 月份 0.92 与之前一年多的控制图，右图是改进后 5 月份开始的控制图。



我：从左右控制图看得出改进后的下限从本来的 0.93 上升到 0.96。除了画图，也可以计算过程能力指数 (Cpk) 反映过程能多满足客户要求 (Cpk 计算公式，详见附件)。

正与你说公司本来要求不能低于 0.93 (客户要求规格)，所以 3 月份之前的能力指数 $Cpk = 1.0$ 。但现在过程收窄了， $Cpk = (1-0.93) / (1-0.96) = 1.75$ 上升了。如果公司因看到提现，也是收窄的规格范围，要求把下限调高到 0.96，你们的能力指数 Cpk 又变回了 1.0。所以 Cpk 可用来代表过程满足客户要求的能力系数。

过程改进组长：从 5 月后数据更新的控制图一直都很稳定，没有异常点，是否表示我们就没有提升的空间，下限 0.96 已经是最佳状态？

我：不一定，你 3 月份分析时，不是说利用二八原则，识别出两个影响最大的板块吗？

现在你的控制图是把所有板块的总数的控制图，建议你试试针对那两个影响最大的板块，画他们的控制图看看。有机会能看到一些异常点。

第二天，过程改进组长：确实看到有较大的波动，但还没有超出上下限，我们会继续观察。

以上实例帮我们了解：

1. 基线

从历史数据得出基线范围能帮助我们区分后面哪些波动是自然噪音，哪些是信号。

信号表示过程可能有基本变化，需要更新基线。控制图帮助我们能做好这区分。

2. 过程能力 (Process Capability)

- 识别。
 1. 过程的声音 (过程的自然波动范围)。
 2. 客户的声音 (客户规格上限下限)。
- 计算过程能力指数 (Cpk) 反映过程能多满足客户要求。

3. 细分

- 不仅仅看总的综合控制图。
- 从总图细分，细分后的控制图可能发现异常点信号 (详见附件有一个细分例子)。

总结

如公司管理者依赖主观判断，定一个“合理”量化目标，反而会影响了团队的改进。更应按团队的历史数据，使用控制图，先制定团队的基线，让团队自己分析根因（也可以配合改进组的辅导），制定纠正措施后，才会有具体的提升目标。

附件

中心极限定理 (Central Limit Theorem CLT)

当总体 (Population) 均值为 μ ，标准差为 σ

随机样本均值

$\overline{X}_n = \frac{\sum_i X_i}{n}$ 的: (n 为每轮抽样样本数量)

均值 = μ

标准差 = $\frac{\sigma}{\sqrt{n}}$

- 样本均值的分布随样本数量增加越来越接近正态分布

检验统计量计算值 $Z_n = \frac{\sqrt{n}(\overline{X}_n - \mu)}{\sigma}$ 接近正态分布。

如何画 **ImR** 控制图

1/ 计算单值 (X) 平均数 (X-bar)

2/ 计算移动极差 (mR moving Range) 均值

3/ 利用以下方程式，计算 X 的上下限

UNPL=X-bar+(2.66*mR 均值)

LNPL=X-bar -(2.66*mR 均值)

4/ 利用以下方程，计算移动极差的上限

URL=3.27* mR 均值

5/ 如果 X 或 mR 有超出范围，表示有过程变化的信号，过程不稳定

Note:

UNPL=Upper Natural Process Limit

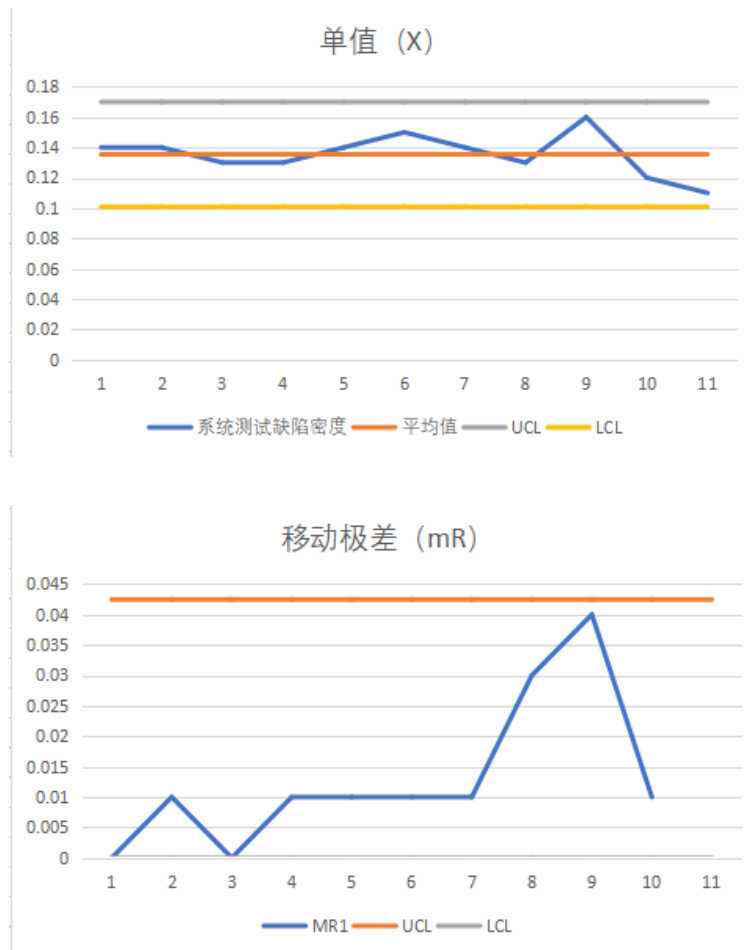
LNPL=Lower Natural Process Limit

URL=Upper Range Limit

例子:

系统测试缺陷密度	移动极差
0.14	0
0.14	0.01
0.13	0
0.13	0.01
0.14	0.01
0.15	0.01
0.14	0.01
0.13	0.03
0.16	0.04
0.12	0.01
0.11	0.01

- 1/ 计算单值 (X) 平均数 ($\bar{X}$) = 0.135 [(0.14+0.14+...+0.11)/11]
- 2/ 计算移动极差 (mR moving Range) 均值如第一移动极差为 0.14-0.14= 0; 第二移动极差为 0.14-0.13= 0.01; 10 点的均值 = 0.0128
- 3/ 计算 X 的上下限: 用上面公式 $UCL=0.135+2.66*0.0128=0.17$, $LCL=0.135-2.66*0.0128=0.10$
- 4/ 计算移动极差的上限用上面公式 $UCL=3.27*0.0128=0.042$, $LCL = 0$
- 5/ 下面 2 图已经加上上下限与平均线:



各种控制图

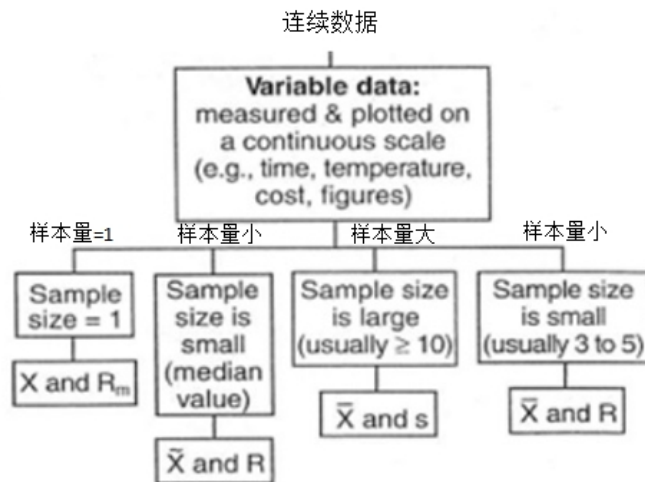
生产线用连续数据的控制图有两部分：

- 均值或中位数
- 标准差或者范围

两个图都要看。通常一些数据本身的值有异常点的话，很可能它的标准差变化也会有异常点。

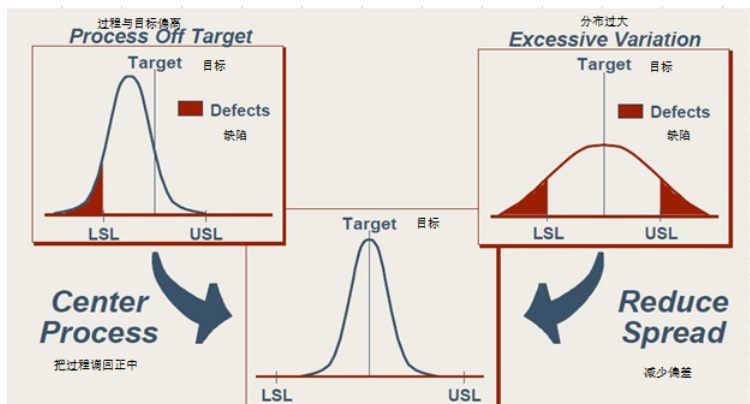
下面就是各种对连续数据的控制图方法，大部分都是按抽样样本的大小制订，比如抽样的样本数大于 10，我们就可以用平均值和标准差来做控制图。

基于样本多少选择恰当的控制图

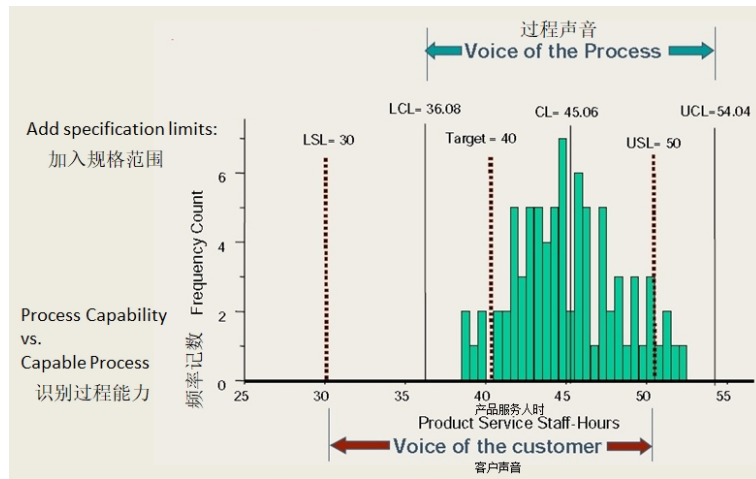


过程能力 (Process Capability)

过程能力反应能否满足客户要求:



中间的图就是客户的声音，USL 和 LSL 是客户要求的规格上下限，右上图就代表过程分布太宽，只有中间部分能满足客户规格要求，左图虽然过程变化范围满足，但偏离了，也不能完全满足客户要求。下图看到过程的范围是不能完全满足客户的规格范围要求:



以下方程式可以计算过程能力指数。 $C_p = \frac{USL - LSL}{6\sigma}$ ，不考虑是否偏移，只考虑分布范围。

如果考虑偏移，计算 $C_{pk} = \min\left[\frac{USL - \mu}{3\sigma}, \frac{\mu - LSL}{3\sigma}\right]$ (选其中最少数值)
指数越大表示过程越能满足客户规格（客户声音）。

参考 References

1. Wheeler, Donald. *Understanding Variation The Key to Managing Chaos*. (2000)
2. Florac, William A. with A. Carleton. *Measuring the Software Process: Statistical Process Control for Software Process Improvement*. (1999) SEI Series in Software Engineering